

שאלה 1 (20 נק')

סעיף א'

הדפסת שורה ריקה על ידי תהליך בן
 הצגת תוכת התיקייה הנוכחית (ביצוע פקודה ls על ידי תהליך בן)
 הדפסת Ending----- בסוף, על ידי תהליך אב.

סעיף ב'

4.1

2.

1, 101

2, 102

2, 101

1, 100

2, 103

2, 100

הסבר: פקודת ה wait תמיד תגרום לאב להמתין לסיום ריצת הבן. הערך של c משוכפל כאשר נקרא fork (עותק של זיכרון האב)

שאלה 2

א. (5 נק') Round Robin (time quantum = 30 ms)

P1	P2	P1	P3	P4	P3	P4	P3
30	50	70	100	130	160	170	210

$$\text{Average turnaround time} = \frac{70+30+110+170}{4} = 95$$

Type equation here.

$$\text{Average waiting time} = \frac{20+10+10+130}{4} = 42.5$$

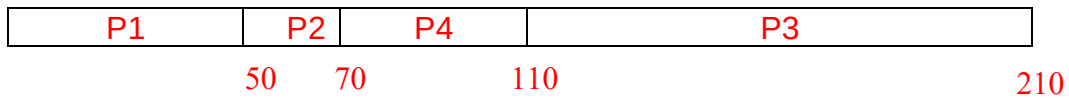
ב. (5 נק') First-Come, First-Served.

P1	P2	P3	P4
50	70	170	210

$$\text{Average turnaround time} = \frac{50+50+130+150}{4} = 95$$

$$\text{Average waiting time} = \frac{0+30+30+110}{4} = 42.5$$

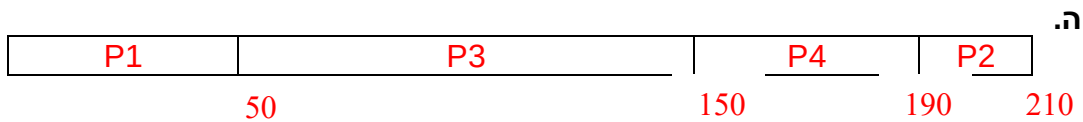
ג. Shortest Job First (5 נק')



$$\text{Average turnaround time} = \frac{50+50+50+170}{4} = 80$$

$$\text{Average waiting time} = \frac{0+30+10+70}{4} = 27.5$$

ד. non-preemptive Priority Scheduling (5 נק')



$$\text{Average turnaround time} = \frac{50+190+110+130}{4} = 120$$

$$\text{Average waiting time} = \frac{0+170+10+90}{4} = 67.5$$

שאלה 3 סעיף א'

sem_init(S1,0) sem_init(S2,0) sem_init(S3,0)		
P1	P2	P3
$C_2 = A_2 + B_2;$ up(S1)	down(S2) $ans_1 = A_1 + B_1 + carry;$ down(S3) $result = ans_1 * 10 + ans_2;$ print(result);	down(S1) $carry = C_2 / 10;$ up(S2) $ans_2 = C_2 \% 10;$ up(S3)

- אפשר גם לפתור את הבעיה בשימוש בסמפור אחד בלבד

הסבר:

P1 צריך לרוץ ראשון כי הוא מחשב סכום של ספרות היחידות של המספרים. לאחר מכן, צריך לרוץ P3 מבצע חיבור של ספרות העשרות ומחשב תוצאה סופית. שימוש לב: P2 יכול לבצע חישוב של ans_1 כבר לאחר חישוב של carry. כמו כן, P2 יכול לבצע חישוב של result רק אחרי ש- ans_2 חושב

סעיף ב'

א. (3 נק') הקוד מקיים מניעה הדדית (mutual exclusion). **כן** / לא

ב. (3 נק') עלול להיווצר קיפאון (deadlock). **כן** / לא

כאשר אין תהליך בקטע הקריטי ערכו של v הוא 0

ג. (3 נק') עלולה להיות הרעבה (starvation). **כן** / לא

יתכן שתהליך מסוים נכשל שוב ושוב בפעולת test-and-set בגלל שתהליכים אחרים

מצליחים שוב ושוב.

ד. (4 נק') נניח כי משנים את האתחול של משתנה v (בשורה 1) ל-1.

מהן התשובות כעת?

i. הקוד מקיים מניעה הדדית (mutual exclusion). **כן** / לא

ii. עלול להיווצר קיפאון (deadlock). **כן** / לא

כעת יתכן גם deadlock משום שכאשר v מאותחל ל 1 - שום פעולת test-and-set אינה

יכולה להצליח

שאלה 4

סעיף א'

ב אינו אפשרי. במקרה של TLB miss, עוברים לחיפוש בטבלת הדפים
 ג אינו אפשרי. TLB hit פירושו שמצאנו דף וכבר אין צורך לחפשו בטבלת הדפים

סעיף ב'

0	1	2	0	3	4	0	1	2
0	1	2	2	3	4	0	1	2
	0	1	1	2	3	4	0	1
		0	0	1	2	3	4	0
				0	1	2	3	4
PF	PF	PF	-	PF	PF	PF	PF	PF

אם כן, במנגנון FIFO יקרו 8 page faults.

במנגנון LRU:

0	1	2	0	3	4	0	1	2
0	1	2	0	3	4	0	1	2
	0	1	2	0	3	4	0	1
		0	1	2	0	3	4	0
				1	2	2	3	4
PF	PF	PF	-	PF	PF	-	PF	PF

אם כן, במנגנון LRU יקרו 7 page faults

במנגנון OPT:

0	1	2	0	3	4	0	1	2
0	1	2	2	3	4	4	4	4
	0	1	1	2	2	2	2	2
		0	0	1	1	1	1	1
				0	0	0	0	0
PF	PF	PF	-	PF	PF	-	-	-

אם כן, במנגנון optimal page replacement יקרו 5 page faults

שאלה 5 סעיף א'

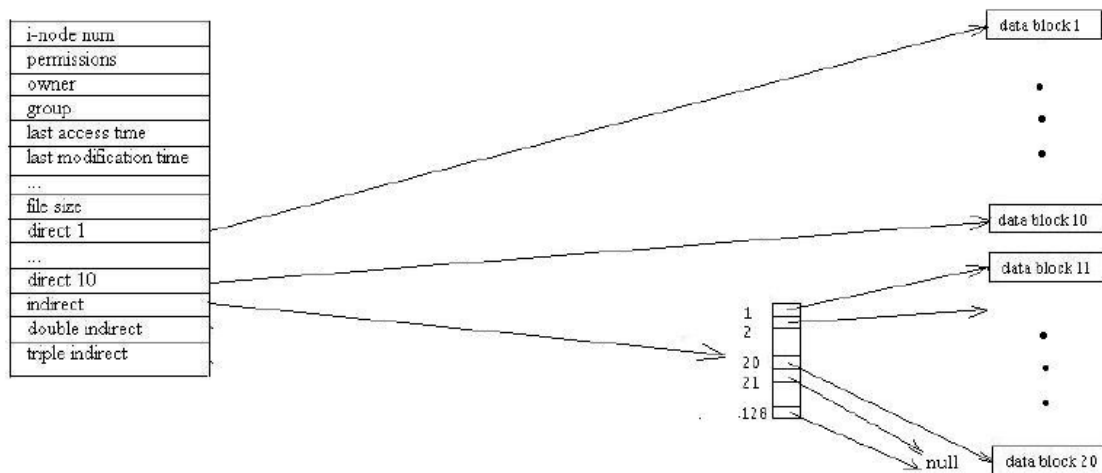
תשובה

10 מצביעים ישירים מצביעים ישירות על 10 בלוקים של נתונים (מ-1 עד 10). נותרו לנו עוד 11 בלוקים, שאחד מהם חייב להיות בלוק פנימי של מערכת הקבצים.

בלוק פנימי של מערכת הקבצים יכול להכיל

$$1\text{kbyte}/8\text{Bytes} = 2^7$$

מצביעים לבלוקים אחרים. במקרה שלנו, אלה בלוקים של נתונים מ-11 עד 20). ראו האיור הבא :



לכן גודל הקובץ יהיה $20 \text{ blocks} * 1 \text{ kbyte} = 20 \text{ kbyte}$

סעיף ב'

המערכת לא נמצאת במצב תקין.

- בלוקים 2 ו-8 חסרים. כדי להחזיר את מערכת הקבצים למצב עקבי, יש להוסיף לרשימת הבלוקים הפנויים.
- בלוק 10 הינו בלוק חופשי ומוקצה. כדי להחזיר את מערכת הקבצים למצב עקבי, יש להוציא את הבלוק מרשימת הבלוקים הפנויים.